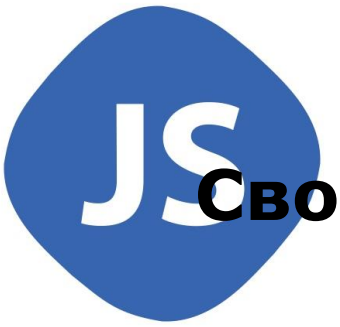


Свойства и методы массивов 2

- Поиск в массиве
- Преобразование массива

Поиск в массиве

Если вам нужно найти **позицию элемента** или **наличие элемента** в массиве, используйте методы **indexOf**() или **includes**() соответственно.

Массивы это вам не строки

При изучении строк мы рассматривали **методы поиска подстрок**. Среди прочих были методы **includes** и **indexOf**, но массивы содержат методы с такими же названиями.

Таким образом, в коде программы равновероятно могут быть следующие конструкции:

- `String.includes(str);`
- `Array.includes(str);`

Внешне очень похоже, но работает это по-разному.

```
// массив из одной строки
let note = ["Acer Extensa 15 EX215-22-R8E3"];
// наличие подстроки в строке
console.log(note[0].includes("Acer")); // true
// наличие элемента в массиве
console.log(note.includes("Acer")); // false
```

Будьте внимательны, проверяйте, **на каком объекте** вызывается метод.

example_1. Тестируем includes на строке

```
<script>
    // инициируем одномерный массив строк
    let note = [
        "Acer Extensa 15 EX215-22-R8E3",
        "Asus VivoBook 15 OLED K513EA",
        "HUAWEI MateBook D 15",
        "Apple MacBook Air 13 2022"
    ];
    // проверяем на вхождение в элементы массива подстроки -Asus-
    // выполняем String.includes()
    console.log(note[0].includes("Asus")); // false
    console.log(note[1].includes("Asus")); // true
    console.log(note[2].includes("Asus")); // false
    console.log(note[3].includes("Asus")); // false
</script>
```

Методы поиска в массиве

Рассмотрим **методы поиска** в массиве.

includes()

Метод **includes**() определяет, содержит ли массив определённый **элемент**, возвращая в зависимости от этого **true** или **false**.

Синтаксис

```
arr.includes(searchElem [, fromIndex = 0]);
```

Параметры

- **searchElem -** искомый элемент.
- **fromIndex** - необязательный. Позиция в массиве, с которой начинать поиск элемента **searchElem**. Значение по умолчанию равно 0.

Возвращаемое значение - **boolean**.

example_2. Тестируем includes на массиве

```
<script>
    // инициируем одномерный массив строк
    let note1 = ["Acer", "Asus", "HUAWEI", "Apple"];
    // проверяем на наличие в массиве note1 элемента -Asus-
    // выполняем Array.includes()
    console.log(note1.includes("Asus")); // true
    //  инициируем одномерный массив строк
    let note2 = [
        "Acer Extensa 15 EX215-22-R8E3",
        "Asus VivoBook 15 OLED K513EA",
        "HUAWEI MateBook D 15",
        "Apple MacBook Air 13 2022"
    ];
    // проверяем на наличие в массиве note2 элемента -Asus-
    // выполняем Array.includes()
    // элемента -Asus- в массиве note2 нет
    console.log(note2.includes("Asus")); // false
</script>
```

Если нужно проверить вхождение подстроки в элементе воспользуемся **методом перебора** элементов массива.

example_3. Использование includes в цикле

```
<script>
    // инициируем одномерный массив строк
    let note = [
        "Acer Extensa 15 EX215-22-R8E3",
        "Asus VivoBook 15 OLED K513EA",
        "HUAWEI MateBook D 15",
        "Apple MacBook Air 13 2022"
    ];
    // проверим каждый элемент массива на вхождение подстроки
    note.forEach(elem => {
        // выполняем String.includes()
        console.log (elem.includes("Asus"));
    });
</script>
```

Выбор способа поиска зависит от поставленной перед вами задачи.

indexOf()

Метод **indexOf**() возвращает **индекс** первого вхождения, по которому данный элемент может быть найден в массиве или -1, если такого индекса нет.

Синтаксис

```
arr.indexOf(searchElem[, fromIndex = 0])
```

Параметры

- **searchElement -** искомый элемент в массиве.

- **fromIndex -** индекс, с которого начинать поиск. Если индекс больше или равен длине массива, возвращается -1, что означает, что массив даже не просматривается. Если индекс является отрицательным числом, он трактуется как смещение с конца массива.

Описание

Метод **indexOf**() сравнивает искомый элемент **searchElem** с элементами в массиве, используя строгое сравнение (оператор ===, тройное равно).

example_4. Тестируем indexOf на массиве

```
<script>
    // инициируем одномерный массив строк
    let note1 = ["Acer", "Asus", "HUAWEI", "Apple"];
    // проверяем наличие элементов в массиве note1
    // выполняем Array.indexOf()
    console.log(note1.indexOf("Asus")); // 1
    console.log(note1.indexOf("Apple")); // 3
    console.log(note1.indexOf("Samsung")); // -1
    // инициируем одномерный массив строк
    let note2 = [
        "Acer Extensa 15 EX215-22-R8E3",
        "Asus VivoBook 15 OLED K513EA",
        "HUAWEI MateBook D 15",
        "Apple MacBook Air 13 2022"
    ];
    // проверяем наличие элемента в массиве note2
    // выполняем Array.indexOf()
    console.log(note2.indexOf("VivoBook")); // -1
```

```
</script>
```

Как и в случае с методом **includes**() воспользуемся **методом перебора** элементов массива.

example_5. Тестируем indexOf в цикле

```
<script>
    // инициируем одномерный массив строк
    let note = [
        "Acer Extensa 15 EX215-22-R8E3",
        "Asus VivoBook 15 OLED K513EA",
        "HUAWEI MateBook D 15",
        "Apple MacBook Air 13 2022"
    ];
    // проверим каждый элемент массива на вхождение подстроки
    note.forEach(elem => {
        // выполняем String.indexOf()
        console.log (elem.indexOf("VivoBook"));
    });
</script>
```

Методы **includes**() и **indexOf**() в **многомерных** массивах.

example_6. Методы includes() и indexOf() в многомерных массивах

```
<script>
    // инициируем двумерный массив
    let note = [
        ["Acer", "14'", "1920x1080", "Swift 3"],
        ["Asus", "16'", "3840x2400", "VivoBook Pro"],
```

```
        ["HUAWEI", "15.6'", "1920x1080", "MagicBook"],
        ["Apple", "13.3'", "2560x1600", "MacBook Air"]
    ];
    // поиск в двумерном массиве
    console.log(note.includes("Apple")); // false
    console.log(note.indexOf("Apple")); // -1
</script>
```

Для поиска в массивах с размерностью два и более используйте метод **find**().

find()

Метод **find**() возвращает значение **первого найденного в массиве элемента**, которое удовлетворяет условию переданному в callback функции. В противном случае возвращается **undefined**.

Синтаксис

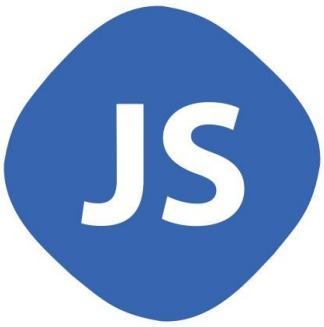
```
let result = arr.find(function(elem, index, array) {
    // ...
});
```

Короткая форма синтаксиса имеет вид:

```
let result = arr.find( elem => {
    // ...
});
```

Параметры

- **function(elem, index, array) -** функция, вызываемая для каждого значения массива, принимает три аргумента:
    - **elem** – текущий обрабатываемый элемент в массиве.



- **index** – необязательный. Индекс текущего обрабатываемого элемента в массиве.
- **array** – необязательный. Массив, по которому осуществляется проход.

Возвращаемое значение

Значение **элемента массива**, если элемент прошёл проверку, иначе **undefined**.

Описание

Метод **find**() вызывает переданную функцию callback один раз для каждого элемента, присутствующего в массиве, **до тех пор, пока она не вернёт true**. Если такой элемент найден, метод **find** немедленно вернёт значение этого элемента. В противном случае, метод **find** вернёт **undefined**.

example_7. Метод find()

```
<script>
    let courses = [
        {id: 1, /* ... */ },
        {id: 2, /* ... */ },
        {id: 3, /* ... */ },
        {id: 4, /* ... */ },
        {id: 5, /* ... */ },
        {id: 6, /* ... */ },
        {id: 7, /* ... */ }
    ];
    // метод находит одно вхождение
    course = courses.find ( item => {
        // если выражение true - вернуть текущий элемент item
        return item.name.includes("JavaScript");
```



```
});
// вывод найденного объекта JS в консоль
console.log(course);
</script>
```

Обратите внимание, условие конца поиска - возврат значения **true**. Именно поэтому в следующем демонстрационном примере будет возвращен **первый объект** из массива.

example_8. Метод find() работает до первого значения true

```
<script>
    let courses = [
        {id: 1, /* ... */ },
        {id: 2, /* ... */ },
        {id: 3, /* ... */ },
        {id: 4, /* ... */ },
        {id: 5, /* ... */ },
        {id: 6, /* ... */ },
        {id: 7, /* ... */ }
    ];
    // находит одно вхождение
    // или возвращает -undefined
    course = courses.find ( item => {
        // на первой же итерации вернем true
        return 1;
    });
    // вывод первого объекта массива
    console.log(course);
```

```
</script>
```

Итого:

- Метод **find**() возвращает **элемент исходного массива**, для которого условие функции даст результат **true**.
- После первого найденного элемента поиск прекращается.

filter()

Метод **filter() создаёт новый массив со всеми элементами**, прошедшими проверку, задаваемую в передаваемой функции.

Синтаксис

```
let result = arr.filter(function(elem, index, array) {
    // ...
});
```

Как обычно, можно записать так:

```
let result = arr.filter( elem => {
    // ...
});
```

Параметры

- **function(elem, index, array) -** функция-предикат, которая будет вызвана для проверки каждого элемента массива. Если функция возвращает **true**, то элемент остаётся в массиве, если **false**, то удаляется. Функция, принимает три аргумента:
  - **elem** – текущий обрабатываемый элемент в массиве.
  - **index** – необязательный. Индекс текущего обрабатываемого элемента в массиве.

- **array** – необязательный. Массив, по которому осуществляется проход.

> ***Примечание***. **Предикатом** называется выражение, возвращающее истину или ложь.

Возвращаемое значение

Вернётся **новый массив с элементами, которые прошли проверку**. Если ни один элемент не прошёл проверку, то будет возвращён пустой массив.

Описание

Метод **filter**() вызывает переданную функцию callback один раз для каждого элемента, присутствующего в массиве, и создаёт новый массив со всеми значениями, для которых функция callback вернула значение, которое **может быть приведено к true**.

example_9. Метод filter()

```
<script>
    // условие отбора
    var price = 25000;
    // находит одно вхождение
    // или возвращает - undefined
    let result = televisions.filter ( item => {
        // проверка вернет либо true либо false
        // если true, текущий item попадет в результирующий
        массив
        return item[6] <= price;
    });
    // вывод результирующего массива
```

```
    console.log(result);
</script>
```

example_10. Метод filter() отбирает все элементы, для которых условие вернет true

```
<script>
    let courses = [
        {id: 1, /* ... */ },
        {id: 2, /* ... */ },
        {id: 3, /* ... */ },
        {id: 4, /* ... */ },
        {id: 5, /* ... */ },
        {id: 6, /* ... */ },
        {id: 7, /* ... */ }
    ];
    // находит все вхождения
    // или возвращает пустой массив -[]
    let res = courses.filter ((item, courses) => {
        // отбираем курсы по JavaScript
        return item.name.includes("JavaScript");
        // отбираем курсы по Assembler
        // return item.name.includes("Assembler");
    });
    // исходный массив
    console.log(courses);
    // результат отбора
    console.log(res);
</script>
```



В следующем демонстрационном примере поработаем с объектом ответа сервера из регионального чемпионата **Профессионалы.** Применим **вложенный** перебор массивов для поиска рейсов, отправляющихся из Москвы.

example_11. Метод filter() во вложенных циклах

```
<script>
    // сохраним массив flights.data.items в переменную items
    let items = flights.data.items;
    // переберем массив items
    items.forEach ((item, items) => {
        // сохраним массив item.flights в переменную flights
        let flights = item.flights;
        // цикл для отбора рейсов
        result = flights.filter( flight => {
            // если рейс выполняется из Москвы,
            // поместии текущий объект массива в результирующий
            массив
            return flight.from.city == "Moscow";
        })
    });
    // результат поиска
    console.log(result);
</script>
```

Обратите внимание, массив **flights.data.items** состоит из одного объекта, но мы все равно перебираем его с помощью метода **forEach**(). Дело в том, что в реальном приложении, в том числе в чемпионате Профессионалы это будет **массив нескольких объектов**.

Итого:

- Метод **filter**() возвращает **элементы исходного массива**, для которых условие функции даст результат **true**.

Преобразование массива

map()

Метод **map**() создаёт новый массив с результатом вызова указанной функции для каждого элемента массива.

Синтаксис

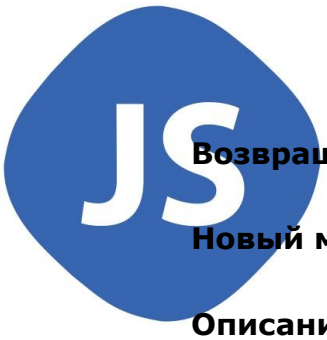
```
let new_array = arr.map(function(elem, index, array) {
    // ...
});
```

Как обычно, можно записать так:

```
let new_array = arr.map( elem => {
    // ...
});
```

Параметры

- **function(elem, index, array) -** функция, вызываемая для каждого значения в массиве, принимает три аргумента:
  - **elem** – текущий обрабатываемый элемент в массиве.
  - **index** – необязательный. Индекс текущего обрабатываемого элемента в массиве.
  - **array** – необязательный. Массив, по которому осуществляется проход.



Возвращаемое значение

Новый массив, где каждый элемент является **результатом** callback функции.

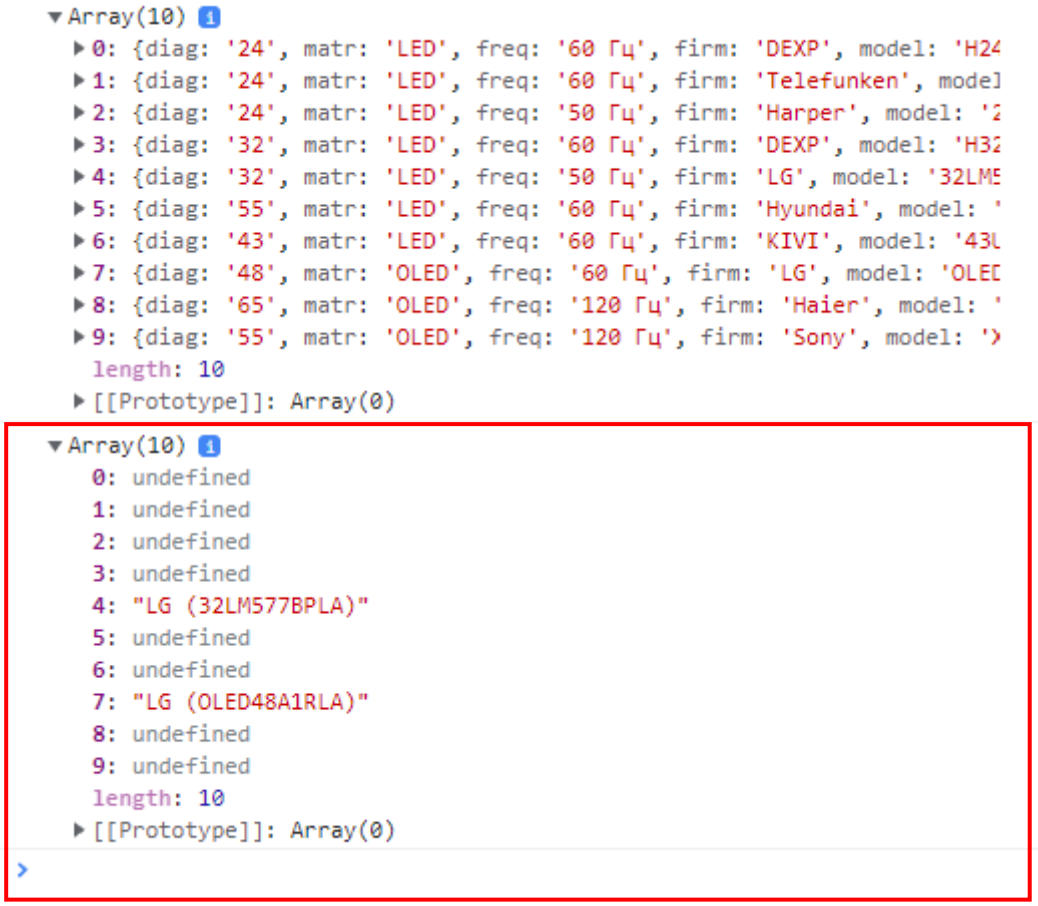
Описание

Метод **map**() вызывает переданную функцию callback один раз для каждого элемента, в порядке их появления и конструирует новый массив из результатов её вызова. Функция callback вызывается только для индексов массива, имеющих присвоенные значения, включая **undefined**. Она не вызывается для пропущенных элементов массива, то есть для индексов, которые никогда не были заданы, которые были удалены или которым никогда не было присвоено значение.

example_12. Метод map()

```
<script>
    // метод map в отличии от forEach возвращает результат
    // результатом будет новый массив
    let res = televisions.map( item => {
        // если фирма LG, то запишем в новый массив сочетание
        данных firm-model
        if (item.firm.includes("LG"))
            return `${item.firm} (${item.model})`;
    });
    // исходный массив
    console.log(televisions);
    // результирующий массив
    console.log(res);
</script>
```

Обратите внимание, размер результирующего массива равен размеру исходного. Для элементов, не соответствующих условию в теле цикла был возвращен результат **undefined**.



Используя метод **map**() создадим **новый массив**, преобразуя его при этом из массива массивов в массив объектов.

example_13. Преобразования массива внутри метода map()

```
<script>

    // инициируем массив массивов

    let teams = new Array(

        [1, "Aerosmith", "США", "1970", ["хард-рок", "блюз-рок",
        "рок-н-ролл", "глэм-метал"]],

        [2, "Pink Floyd", "Великобритания", "1965",
        ["психоделический-рок", "арт-рок", "эйсид-рок",
```

```
        "прогрессивный-рок"]],
        [3, "The Beatles", "Великобритания", "1960", ["рок",
        "рок-н-ролл", "поп", "хард-рок"]],
        [4, "AC/DC", "Австралия", "1973", ["хард-рок", "блюз-
        рок", "рок-н-ролл"]],
        [5, "Scorpions", "ФРГ", "1965", ["хард-рок", "глэм-
        метал", "хеви-метал"]],
        [6, "Ленинград", "Россия", "1997", ["рок", "ска", "фолк",
        "панк", "панк-рок"]]
);
// создадим копию массива, в которой
// вложенные массивы заменим на вложенные объекты
res = teams.map ( item => {
    // вернем объект
    return {
        "id" : 10 + item[0],
        "name" : item[1],
        "country" : item[2],
        "year" : item[3],
        // приведем массив третьего уровня к строке
        "style" : item[4].join()
    }
});
// исходный массив
console.log(teams);
// результирующий массив
console.log(res);
```

```
</script>
```

В следующем демонстрационном коде проявим великодушие негоцианта и создадим новый массив, в котором **уменьшим цену каждого телевизора** из подключенного массива на 10%.

> ***Важно.*** То, что сейчас будет сделано, приблизит вас еще на один шаг к клану избранных. В миру их еще называют **программистами**.

example_14. Метод map() и негоцианство

```
<script>
    // установим скидку в размере 10% на каждый телевизор
    results = televisions.map ( item => {
        // меняем цену каждого телевизора
        item[6] = item[6] - (item[6] * 0.1);
        // возвращаем элемент
        return item;
    });
    // исходный массив
    console.log(televisions);
    // измененный массив
    console.log(results);
</script>
```

Мы сделали широкий жест. Но... Посмотрите, цены изменились не только в результирующем массиве, но и в исходном. Это не то, что нам бы хотелось.

Широкий жест — это хорошо, но как люди узнают о **нашем величии**, если не будет с чем сравнивать? Исходный массив желательно оставить в неизменном состоянии.



Правильно в таком случае сделать копию элемента массива, и уже в копию внести требуемые изменения.

example_15. Создание копий массива внутри преобразований

```
<script>

    // установим скидку в размере 10% на каждый телевизор

    results = televisions.map ((item, index, televisions) => {

        // сделаем срез массива item

        newArr = item.slice(0,);

        // меняем цену для телевизора нового массива

        newArr[6] = item[6] - (item[6] * 0.1);

        // возвращаем новый массив
```

```
        return newArr;
    });
    // исходный массив
    console.log(televisions);
    // измененный массив
    console.log(results);
</script>
```

Вот теперь **console.log** свидетель нашего величия.

Итого:

- Метод **map**() возвращает новый **массив**.
- **Размер** результирующего массива равен размеру исходного.

sort()

Метод **sort()** — сортирует массив на месте, меняя в нём порядок элементов. Он также возвращает отсортированный массив, но обычно возвращаемое значение игнорируется, так как изменяется сам **исходный массив**.

Синтаксис

```
arr.sort([compareFunction])
```

Параметры

- **compareFunction** – необязательный параметр. Указывает **функцию**, **определяющую порядок сортировки**. Если опущен, массив сортируется в соответствии со значениями кодовых точек каждого символа **Unicode**, полученных путём **преобразования каждого элемента в строку**.

Важно. По умолчанию элементы сортируются как **строки**.

Возвращаемое значение

Отсортированный массив. Важно, что копия массива **не создаётся** - массив сортируется на месте.

Описание

Если функция сравнения **compareFunction** не предоставляется, элементы сортируются путём **преобразования их в строки и сравнения строк в порядке следования кодовых точек Unicode**.

example_16. Простая сортировка одномерного массива

```
<script>
    // инициируем исходный массив
    var arr = ['e', 2, 10, 'g', '7', 'д', undefined];
    // сортируем массив
    arr.sort();
    // порядок сортировки:
    // 1. цифры
    // 2. символы En
    // 3. символы Ru
    console.log(arr);
</script>
```

example_17. Сортировка строковых значений одномерного массива

```
<script>
    // одномерный массив для сортировки
```

```
    let note = [
        "The Lady of the Lake",
        "451' по Фаренгейту",
        "Москва - Петушки",
        "A Midsummer Night's Dream",
        "Чемодан",
        "1984"
    ];
    // исходный массив
    console.log(note);
    // сортируем массив
    note.sort();
    // отсортированный массив
    console.log(note);
</script>
```

Примечание. Если вы не владеете концепцией функций, следующие примеры могут оказаться сложными, по этой причине их можно пропустить. Вернемся к ним после изучения пользовательских функций.

Двумерный массив сортировать так просто не получится. Для сортировки придется составить собственный алгоритм сравнения элементов.

Чтобы использовать собственный алгоритм сортировки, нужно предоставить **собственную функцию** в качестве аргумента. Для каждой пары значений функция должна возвращать результат по **следующему правилу**:

- `if (a > b) return 1; // если первое значение больше второго`
- `if (a == b) return 0; // если равны`
- `if (a < b) return -1; // если первое значение меньше второго`



Например, в демонстрационном примере **example_18** сортируем массив по значению его первых элементов (чисел).

example_18. Сортировка двумерных массивов

```
<script>
    // двумерный массив для сортировки
    let note = [
        [6, "1984"],
        [4, "A Midsummer Night's Dream"],
        [1, "The Lady of the Lake"],
        [3, "Москва - Петушки"],
        [5, "Чемодан"],
        [2, "451' по Фаренгейту"]
    ];
    // сортируем массив с указанием правила сортировки
    // передаем два параметра-массива
    note.sort( (el1, el2) => {
        // производим сравнение по первым элементам массивов
        if (el1[0] > el2[0]) return 1;
        if (el1[0] === el2[0]) return 0;
        if (el1[0] < el2[0]) return -1;
    });
    // результат сортировки
    console.log(note);
</script>
```

Сортировать можно и по значению любых других элементов. Например, так, как продемонстрировано в примере **example_19**.

example_19. Сортировка двумерных массивов по произвольному элементу

```
<script>
    // двумерный массив для сортировки
    let note = [
        [6, "1984"],
        [4, "A Midsummer Night's Dream"],
        [1, "The Lady of the Lake"],
        [3, "Москва - Петушки"],
        [5, "Чемодан"],
        [2, "451' по Фаренгейту"]
    ];
    // сортируем массив с указанием правила сортировки
    // передаем два параметра-массива
    note.sort( (el1, el2) => {
        // производим сравнение по вторым элементам массивов
        if (el1[1] > el2[1]) return 1;
        if (el1[1] === el2[1]) return 0;
        if (el1[1] < el2[1]) return -1;
    });
    // результат сортировки
    console.log(note);
</script>
```

Правильное сравнение строк

Использование арифметических операторов при сравнении строк допустимо, но не всегда корректно. Дело в том, что такое сравнение не учитывает особенности национальных символов.



Правильный алгоритм сравнения строк сложнее, чем может показаться, так как **разные языки используют разные алфавиты**. Поэтому браузеру нужно знать, какой язык использовать для сравнения.

Все современные браузеры поддерживают **стандарт** ECMA 402, обеспечивающий правильное сравнение строк на разных языках с учётом их правил.

Для этого существует **соответствующий метод**.

Вызов str**.localeCompare(**str2**)** возвращает число, которое показывает, какая строка больше **в соответствии с правилами языка**:

- **Отрицательное** число, если str меньше str2.
- **Положительное** число, если str больше str2.
- **0**, если строки равны.

example_20. Правильная сортировка строк-элементов массива

```
<script>
    // двумерный массив для сортировки
    let note = [
        [6, "1984"],
        [4, "A Midsummer Night's Dream"],
        [1, "The Lady of the Lake"],
        [3, "Москва - Петушки"],
        [5, "Чемодан"],
        [2, "451' по Фаренгейту"]
    ];
    // сортируем массив с учетом национальных кодировок
    note.sort( (el1, el2) => {
        return el1[1].localeCompare(el2[1])
    });
```

```
    // результат сортировки
    console.log(note);
</script>
```

Сравните вывод ***неправильного*** и ***правильного*** сравнения строк из демонстрационных примеров **example_19** и **example_20**.

> ***Важно.*** Это далеко не все методы, но представленных методов и возможностей вполне хватит для начинающего веб-разработчика.

В **самостоятельных работах** к этому уроку:

- Поиск ошибки вывода.
- Фильтрация элементов трехмерного массива.
- Поиск значений массивов и объектов в сложных структурах данных.
- Изменение копий массивов в методах перебора элементов.
- Сортировка вложенных объектов массива.

P.S.

Для отработки и закрепления учебного курса **донам** группы предоставляется следующий раздаточный материал.

К каждому уроку курса:

- Файлы **демонстрационного кода** (example);
- **Задачи** с решениями в контексте рассматриваемых вопросов урока (task).

К каждой теме курса:

- **Практические** работы